Structured Outputs

Deep Dive into SGLang — Chapter 15

Yin Yang

Foundation Books Project

Where this chapter sits

A response can be one token away from invalid JSON.

- Repairing the text *after* sampling would change the distribution the server claimed to serve.
- So validity must move into token choice — the grammar removes illegal tokens *before* the sampler draws.
- This is the structured-output sibling of the sampling chapter: it builds the masks that sampling then applies.

Goal: by the end you can trace one constrained request from its JSON schema, through a grammar mask, to a committed token — and name the row that owns every step.

Roadmap

The constrained-decoding problem

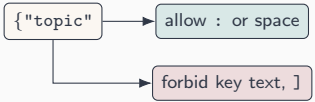
Readiness, masks, and invariants

The boundary and the evidence

The constrained-decoding problem

The running example: a JSON pancake request

$r_{\text{pan}}^{\text{json}}$ on the same Gemma service asks for an object with fields `topic`, `setup`, `punchline`.



- After a completed key, the grammar may allow a colon or whitespace.
- It must *not* allow a token that continues the key, or an unrelated closing bracket.

Validity is a row-local decoding decision, not a postprocessing rule.

Three objects: grammar object, grammar state, token mask

Grammar object the compiled form of the request constraint (schema, regex, EBNF, or structural tag) — like a compiled regex.

Grammar state $q_r(t)$ that object's request-local automaton position after t committed tokens — the pointer inside the automaton.

Token mask m_q the vocabulary-indexed validity vector derived from the state and applied to the logits row — the legal next-token set.

Object is compiled once; state is per-request; mask is per-step.

The constrained sampling step, in one formula

For request r with grammar state $q_r(t)$ and allowed set $A_r(t) \subseteq \mathcal{V}$:

$$\ell'_{r,v} = \begin{cases} \ell_{r,v}, & v \in A_r(t), \\ -\infty, & v \notin A_r(t), \end{cases} \qquad m_{q_r(t)}[v] = 1 \iff v \in A_r(t).$$

- Mask first, then run the *ordinary* sampler on ℓ'_r.
- Once token $y_{r,t}$ is committed, advance the state to $q_r(t+1)$.

An $-\infty$ on forbidden columns — the same additive move as a logit bias or a min-new-token ban, but supplied by the grammar.

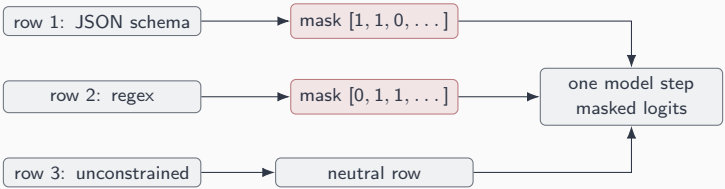
Readiness, masks, and invariants

Two phases: gate at admission, mask at every token

1. **Readiness gate.** A constrained request cannot enter an executable batch until its grammar object is ready — and *every* rank agrees it is ready.
2. **Per-token commit.** On each step: fill a mask row per constrained request, move it to the device, apply it, sample, then advance the same grammar object on the committed token.

A request can be KV-feasible yet not runnable — because grammar compilation has not finished.

Different grammars, one batch — masks stay row-local



Three rows share one forward pass; each carries its own grammar state and mask. An unconstrained or finished row is neutral — it masks nothing.

Lose row alignment and a valid JSON mask lands on another request's logits.

The one rule: structured-prefix alignment

Invariant (Structured-prefix alignment)
A constrained step is valid only when request identity, grammar readiness, committed prefix, generation mode, row and commit ownership, lifetime, backend mask operation, and the sampled token all describe the same structured-output state.

A valid vocabulary token can still be wrong — if it was masked from another request's schema, a stale prefix state, or a row that moved during batch filtering. The mask must describe exactly the committed prefix.

From concept to code: mask, apply, and advance

One method fills every constrained row's mask, then moves it to the device.

from `python/sglang/srt/sampling/sampling_batch_info.py`

```
def update_regex_vocab_mask(self):
    if not self.grammars:
        self.vocab_mask = None; self.apply_mask_func = None; return
    first_grammar = next(grammar for grammar in self.grammars if grammar)
    self.vocab_mask = first_grammar.allocate_vocab_mask(...)
    self.apply_mask_func = first_grammar.apply_vocab_mask
    for i, grammar in enumerate(self.grammars):
        if grammar and not grammar.finished and not grammar.is_terminated():
            grammar.fill_vocab_mask(self.vocab_mask, i)
    self.vocab_mask = first_grammar.move_vocab_mask(self.vocab_mask, self.device)
```

Row *i* fills mask row *i* — and only if its grammar is present, unfinished, and not terminated. Finished rows stay neutral.

The boundary and the evidence

The grammar-mask contract: what is inside, what is downstream

Inside the contract

- compile / copy the constraint
- maintain matcher state
- fill and apply the valid-token mask
- accept or roll back committed tokens

Outside / downstream

- attention layout, KV allocation
- temperature, top-k/p, penalties
- whether text executes a tool
- argument / protocol validation

Structured outputs make token choices legal; parsers and protocol handlers interpret the legal text.

What the measurements must separate

Constraint cost hides in four different places:

Phase	What it costs
Grammar preparation	compile or cache lookup, queue wait
Per-token masking	mask fill, mask application
Deterministic jump-forward	retokenization, rollback
Failed compilation	timeout aborts

Higher tok/s need not mean a cheaper mask — it can just mean fewer or shorter constrained decode steps. Attribution is what makes the number honest.

What to carry forward

1. Structured output is a **sampling-boundary contract**, not a repair pass: mask before the draw, never edit after.
2. Three objects — **grammar object, grammar state, token mask** — and the constrained step is an additive $-\infty$ on forbidden columns.
3. **Readiness** gates admission and is **rank-agreed**; **commit** advances the grammar only on tokens the runtime kept.
4. **Structured-prefix alignment** is the one invariant: a valid token is still wrong if its mask, row, or prefix belongs to someone else.
5. Every concept has a **home in** `constrained/` and the bitmask kernels; that link is the spine of this book.

Next: Chapter 16 takes the syntactically valid text this chapter produces and asks what it means — reasoning spans and tool calls.